

TokenScope: Token-Level Explainability and Interpretability for Code-Oriented Tasks in Large Language Models

Amirreza Esmaili

a.esmaeili@ubc.ca

University of British Columbia
Kelowna, British Columbia, Canada

Fatemeh Fard

fatemeh.fard@ubc.ca

University of British Columbia
Kelowna, British Columbia, Canada

Abstract

Understanding how Large Language Models (LLMs) make token-level decisions during code generation remains a major challenge for both researchers and practitioners. While recent tools provide insights into model internals or generation outcomes, they often lack decoding-time signals, fine-grained uncertainty measures, and interactive mechanisms for exploring alternative generation paths. We present TokenScope, an interactive interpretability and analysis tool for decoder-based LLMs that exposes token-level metrics, attention patterns, and structural information during generation. TokenScope supports interactive token replacement, counterfactual branching, and code-aware aggregation via abstract syntax trees. By unifying decoding-time signals with structural program analysis, TokenScope enables systematic investigation of LLM behaviour during code generation.

CCS Concepts

• **Computing methodologies** → *Natural language generation; Information extraction; Lexical semantics; Causal reasoning and diagnostics*; • **Software and its engineering**;

Keywords

Large Language Models, Code Generation, Interpretability, Explainability, Attention Analysis, Program Analysis

ACM Reference Format:

Amirreza Esmaili and Fatemeh Fard. 2018. TokenScope: Token-Level Explainability and Interpretability for Code-Oriented Tasks in Large Language Models. In *Proceedings of Companion Proceedings of the 34th ACM Symposium on the Foundations of Software Engineering (FSE '26)*. ACM, New York, NY, USA, 12 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

Large Language Models (LLMs) based on transformer decoders have become central to modern software engineering workflows, supporting tasks such as code generation, refactoring, test synthesis, and program understanding [3, 6, 8]. Despite their effectiveness, these models remain opaque: practitioners and researchers are often unable to explain *why* a model produced a particular token,

how uncertain it was at different points in the generation, or *what alternatives* were plausibly considered but discarded. This lack of explainability is particularly problematic in code generation, where small token-level deviations can cascade into syntactic errors, semantic bugs, or incorrect API usage.

Existing interpretability and explainability tools provide partial insight into LLM behaviour but leave important gaps. Prior work largely emphasizes dataset-level analysis, post-hoc evaluation, internal representation inspection, or static attention visualization, often requiring offline analysis and substantial expertise [1, 4, 5, 7, 9]. These approaches rarely expose decoding-time uncertainty, alternative token candidates, or decision margins, and they typically treat generation as a single outcome rather than as a sequence of interdependent token-level decisions. As a result, researchers lack the infrastructure to study where uncertainty arises during generation, how attention and probability mass shift over time, and how local decisions propagate into structural errors in code.

To address these challenges, we introduce **TokenScope**, an interactive explainability and interpretability environment for decoder-based LLMs, with a focus on code generation. TokenScope instruments the decoding process itself, capturing fine-grained token-level signals, including confidence measures, margin confidence, entropy, token surprisal, alternative candidates, and attention weights, at each generation step. These signals are streamed to an interactive frontend that enables direct inspection and comparison of token-level decisions.

TokenScope supports both researchers and practitioners by enabling interactive exploration of LLM behaviour during code generation. Users can apply modifications at any point in a generation by replacing a token with an alternative candidate or custom text and continuing generation from the modified prefix, allowing rapid exploration of counterfactual outputs and failure modes. By aligning decoding-time uncertainty and attention signals with code structure via Abstract Syntax Tree (AST) analysis, TokenScope helps practitioners diagnose where confidence degrades, identify structurally fragile regions of generated code, and trace how local token-level decisions propagate into higher-level program behaviour. Together, these capabilities provide a practical environment for debugging, auditing, and understanding LLM-generated code beyond post-hoc inspection.

This paper makes the following contributions:

- we introduce TokenScope, an interactive tool that exposes decoding-time token-level signals for transformer-based LLMs, with a focus on code generation;
- we provide integrated support for token-level uncertainty metrics, attention saliency, and interactive branching for counterfactual generation;

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

FSE '26, Montreal, Canada

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/2018/06

<https://doi.org/XXXXXXX.XXXXXXX>

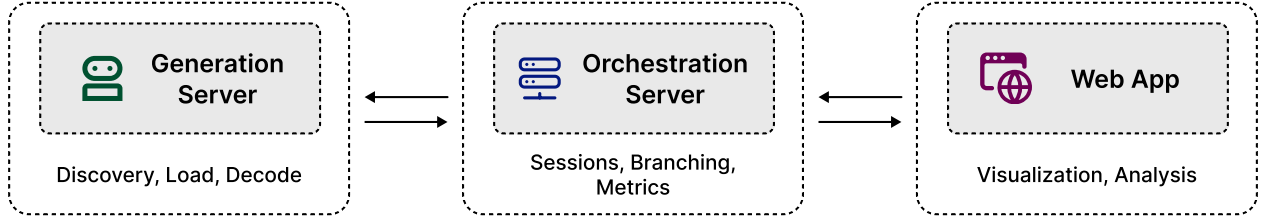


Figure 1: Overview of TokenScope’s modular architecture. The generation server exposes decoder-only LLMs, the orchestration server orchestrates generation, metrics, and branching, and the web-based frontend application provides interactive exploration of decoding-time signals.

- we enable code-aware analysis by aligning token-level signals with AST entities at multiple levels of granularity; and
- we present a unified environment that combines explainability, interpretability, and user-driven exploration, addressing limitations of existing tools that focus on post-hoc analysis or isolated internal signals.

2 Background

A range of tools has been proposed to improve the interpretability and explainability of transformer-based models. We briefly review the most relevant categories and highlight how TokenScope differs.

General-purpose interpretability platforms such as LIT [9] and Phoenix [2] provide interfaces for dataset inspection, model comparison, and embedding analysis. While valuable for evaluation and debugging at the dataset level, these tools do not expose decoding-time token probabilities, alternative candidates, or uncertainty metrics during generation.

Mechanistic interpretability frameworks such as TransformerLens [7] and sparse autoencoder approaches [5] enable detailed inspection of attention heads, neurons, and internal activations. These approaches offer powerful analytical capabilities but typically require offline analysis and substantial technical expertise, and they are not designed for interactive exploration of individual generations.

Visualization tools such as Ecco [1] and Attention Analysis [4] focus primarily on attention weights. While useful for understanding information flow, attention alone is an incomplete proxy for model confidence or decision-making, and these tools do not integrate uncertainty measures or structural program analysis.

3 System Overview

TokenScope is an interactive explainability and interpretability system for decoder-only transformer LLMs, designed to expose fine-grained token-level signals during text and code generation. The system captures internal model statistics at decoding time and presents them through an interactive web interface that supports inspection, comparison, and counterfactual exploration of generation trajectories.

TokenScope follows a modular three-component architecture, enabling support for arbitrary decoder-only models via custom generation server implementations, as illustrated in Figure 1.

- (i) The generation server is responsible for model discovery, loading, and low-level decoding, exposing Hugging Face-compatible decoder-only models through a minimal generation API.
- (ii) The orchestration server coordinates generation requests through the generation server, manages user sessions, branching generation states, metric computation and aggregation, and frontend-facing API endpoints.
- (iii) The web-based frontend, implemented in React, provides interactive visualizations and controls for exploring model behaviour.

Generation is performed incrementally and streamed to the frontend. At each decoding step, TokenScope records token probabilities, alternative candidates, attention weights, and derived uncertainty metrics, associates them with the generated token, and makes them available for interactive visualization and analysis (Figure 2).

```
01 def sort(l):
02     for i in range(len(l)):
03         for j in range(i+1,len(l)):
04             if l[i]>l[j]:
05                 l[i],l[j]=l[j],l[i]
06     return l
07 l=[1,2,3,4,5,6,7,8,9,10]
08 print(sort(l))
```

Figure 2: Output generated by Qwen2.5 Coder 1.5B Base using greedy generation, when given ‘def sort(l):’ as input. Grey represents the prompt, while other token colors represent token-level confidence scores in an increasing order of Red, Orange, Dark Yellow and Black.

Token rank is defined as the position of y_t when candidate tokens are sorted in descending order of probability.

Token Surprisal. Token surprisal measures the unexpectedness of a generated token:

$$\text{Surprisal}(y_t) = -\log p_t(y_t) \quad (2)$$

Intuitively, this metric measures how unexpected a generated token is given the preceding context.

Margin Confidence. Margin confidence captures the difference between the most likely and the second most likely tokens. This metric reflects local decisiveness rather than absolute correctness.

Entropy. Token-level entropy quantifies uncertainty over the candidate distribution:

$$H_t = -\sum_v p_t(v) \log p_t(v) \quad (3)$$

Sequence Perplexity. Sequence perplexity over a generated sequence $y_{1:T}$ is defined as

$$\text{Perplexity} = \exp\left(\frac{1}{T-1} \sum_{t=2}^T -\log p_t(y_T)\right) \quad (4)$$

which quantifies the average unpredictability of the full generated sequence, where lower values indicate the model finds the sequence more probable.

Attention Mass. For a generated token t_i , attention mass is defined as the average amount of attention it receives from all subsequently generated tokens t_j where $j > i$. Formally, for each attention head h ,

$$AM_h(t_i) = \frac{1}{N-1} \sum_{j=i+1}^N A_h(j, i), \quad (5)$$

where $A_h(j, i)$ denotes the attention weight from token t_j to token t_i in head h .

Intuitively, attention mass measures how much a token continues to influence later decoding decisions.

6 Code-Aware Structural Analysis

In Code Analysis Mode, TokenScope aligns generated tokens with AST entities using Tree-Sitter parsers.

6.1 Token–Entity Alignment and Aggregation

LLM token boundaries do not generally align with AST entities, as tokenization is independent of syntactic parsing. An AST entity is considered a candidate for an LLM token if the token is fully contained within the entity span, overlaps with the entity start, or overlaps with the entity end.

When multiple entities are candidates for a single token, TokenScope assigns the entity with the highest predefined priority, reflecting semantic importance (e.g., identifiers over delimiters).

6.2 Code Analysis Modes

To support analysis at different semantic resolutions, the tool provides five AST entity modes, each defining a distinct aggregation level for token-level metrics and attention signals.

Token Mode represents the finest granularity, where each generated token is analyzed independently. This mode is useful for

inspecting localized uncertainty, abrupt probability drops, or attention shifts at specific lexical positions.

Expression Mode groups tokens into syntactic expressions such as literals, binary operations, function calls, or indexing expressions. Aggregating metrics at this level highlights uncertainty and attention patterns within semantic units that often correspond to logical decisions in code.

Statement Mode aggregates tokens belonging to complete statements, including assignments, control-flow statements, and return statements. This mode enables comparison of confidence and attention across executable units that affect program behaviour.

Line Mode aggregates tokens belonging to a single line of code, as determined by source-level line boundaries after formatting. This mode enables analysis of uncertainty and attention at a granularity commonly used by developers when reading and debugging code, and supports identifying lines that disproportionately contribute to low-confidence generations or error propagation.

Block Mode groups tokens within structural blocks such as function bodies, loops, or conditional branches. Analysis at this level supports studying how uncertainty and attention evolve across larger scopes and control-flow regions.

Together, these five modes allow TokenScope users to move seamlessly between fine-grained token inspection and higher-level structural analysis, supporting both detailed debugging and holistic evaluation of generated code.

7 Limitations

TokenScope currently targets decoder-only autoregressive models and requires access to token probabilities and attention weights, which limits its applicability to closed or heavily abstracted LLM APIs. Its code-aware analysis relies on Tree-Sitter parsers and is presently limited to Python; while support for additional languages is feasible, structural analysis depends on parser availability and may fail on incomplete or syntactically invalid generations. Finally, TokenScope is designed as an evaluation and analysis tool rather than a production inference system, and the overhead of extracting, aggregating, and transferring fine-grained interpretability signals prevents it from matching the performance of deployment-oriented inference frameworks.

8 Conclusion

We presented TokenScope, an interactive tool for observing, analyzing, and manipulating token-level behaviour in decoder-based LLMs for code generation. By exposing decoding-time uncertainty metrics, attention signals, interactive branching, and code-aware structural aggregation, TokenScope enables analyses that are difficult to perform with existing tools. TokenScope can support both practical debugging and empirical research on LLM behaviour in software engineering. TokenScope’s source code is open in order to facilitate further study in this direction.

Data Availability

TokenScope’s source code is open and available on Github¹.

¹<https://github.com/Amirresm/tokenscope>

References

- [1] J Alammr. 2021. Ecco: An Open Source Library for the Explainability of Transformer Language Models. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing: System Demonstrations*. Association for Computational Linguistics.
- [2] Arize-ai. 2025. *Phoenix*. Arize-ai. <https://github.com/Arize-ai/phoenix>
- [3] Mark Chen. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).
- [4] Kevin Clark, Urvashi Khandelwal, Omer Levy, and Christopher D. Manning. 2019. What Does BERT Look At? An Analysis of BERT’s Attention. In *Black-BoxNLP@ACL*.
- [5] Alan Cooney. 2023. Sparse Autoencoder Library. https://github.com/ai-safety-foundation/sparse_autoencoder.
- [6] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, et al. 2022. Competition-level code generation with alphacode. *Science* 378, 6624 (2022), 1092–1097.
- [7] Neel Nanda and Joseph Bloom. 2022. TransformerLens. <https://github.com/TransformerLensOrg/TransformerLens>.
- [8] Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Romain Sauvestre, Tal Remez, et al. 2023. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950* (2023).
- [9] Ian Tenney, James Wexler, Jasmijn Bastings, Tolga Bolukbasi, Andy Coenen, Sebastian Gehrmann, Ellen Jiang, Mahima Pushkarna, Carey Radebaugh, Emily Reif, and Ann Yuan. 2020. The Language Interpretability Tool: Extensible, Interactive Visualizations and Analysis for NLP Models. 107–118 pages. <https://www.aclweb.org/anthology/2020.emnlp-demos.15>

A Tool Availability

TokenScope is fully open-source. The source code is publicly available at <https://github.com/Amirresm/tokenscope>. To support reproducibility and ease of deployment, the repository includes a *Dockerfile* that enables consistent setup across environments.

B Screencast

A screencast demonstrating TokenScope is available at <https://youtu.be/HI1L1X9LruQ>. This video is also available for download ².

C Tool Walkthrough

This section provides an illustrative walkthrough of TokenScope for a simple code generation scenario. In this scenario, the base variant of Qwen 3 0.6B model is used to continue the input `def sort(1):`, effectively generating a sort function in a code completion setting.

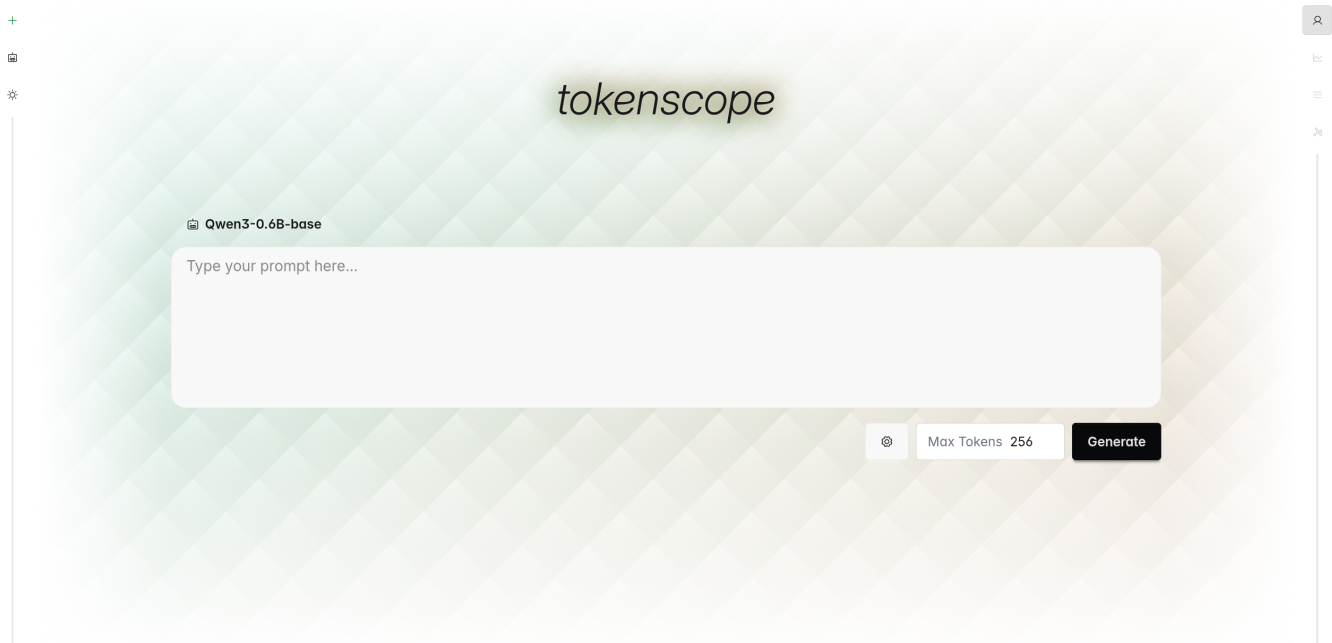


Figure 6: Users choose the model, prompt and generation settings on the landing page.

C.1 Landing Page

Figure 6 presents the home page. User starts by loading a model (left on Figure 7), entering a prompt, configuring the generation settings (right on Figure 7) and selecting the **generate** button.

C.2 Main Content View

When generation starts, the user is redirected to the main content view (Figure 8), where the generated output and a high-level summary of the current generation metrics and metadata are visible.

Selecting the **View Trends Chart** on the *Statistics* sidebar opens the trends chart (Figure 9), which provides an overview of all metrics per token across the generation.

Selecting an individual token switches to the **Token Details** sidebar (Figure 10), presenting the confidence score and the token's string (with whitespace characters such as new line visualized), special token types, token metrics and alternative tokens.

With an individual token selected, the user can switch to the **Attention Details** sidebar (Figure 11) to examine the token's attention-relevant metrics such as *attention mass*, attention source tokens (Figure 12) and subsequent attending tokens (Figure 13).

C.3 Generation Branching

When an alternative token is selected in the *Token Details* sidebar, a branch in the generation is created, and the generation is continued with the new prefix, which is managed in the **Graph View**, as illustrated in Figure 14.

²<https://raw.githubusercontent.com/Amirresm/tokenscope/main/supplementary/demo/demo.mp4>

Current Model

Name: Qwen3-0.6B-base

Source: transformers

Path: /storage/c/ai/models/llm/Qwen/Qwen3-0.6B-base

Available Models

qwen3	
/storage/c/ai/models/llm/Qwen/Qwen3-0.6B	Load
/storage/c/ai/models/llm/Qwen/Qwen3-0.6B-base	Load
/storage/c/ai/models/llm/Qwen/Qwen3-4B	Load
/storage/c/ai/models/llm/Qwen/Qwen3-4B-base	Load
/storage/c/ai/models/llm/Qwen/Qwen3-VL-2B-Instruct-FP8	Load
/storage/c/ai/models/llm/Qwen/Qwen3-VL-8B-Instruct-FP8	Load

Token Level View Config

Alternative Tokens

5

Top K

1

Top P

1

Temperature

1

Attention Layer ?

-1

Attention Top N

10

Figure 7: Left: The model management modal displays the currently loaded and all available models. Right: The generation settings drop-down allows defining the number of alternative tokens, sampling settings and attention configuration.

C.4 Code Analysis

After selecting a region of the generation corresponding to a code snippet, TokenScope analyses, extracts, and assigns code entities to the generated tokens. These entities are presented in the **Code Analysis** view, shown in Figure 15. The **Code Analysis Statistic** sidebar presents a summary of extracted entities and their average confidence scores. This summary is also provided in a bar plot format (Figure 16).

Additionally, TokenScope extracts attention relationships between code entities and presents them as a heatmap, as shown in Figure 17.

Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009

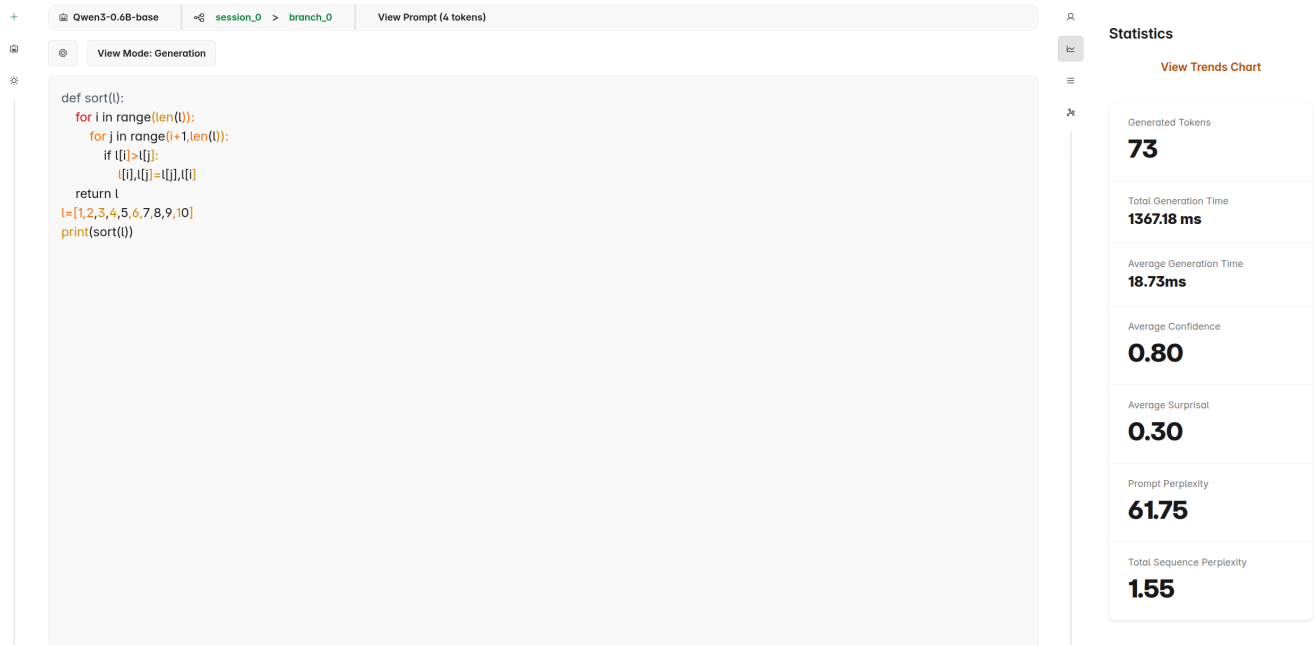


Figure 8: The main content view displays the generated output and a high-level summary of the current generation metrics and metadata.

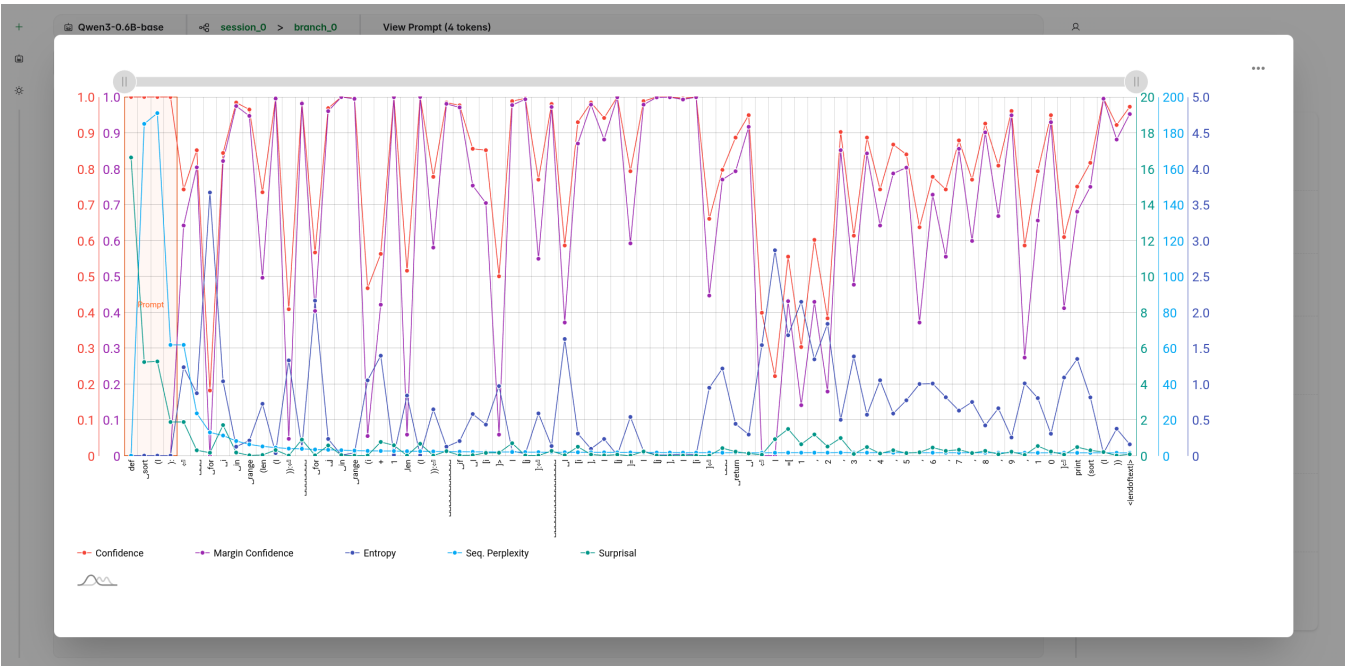


Figure 9: Trends chart displays an overview of all metrics for each token across the generation.

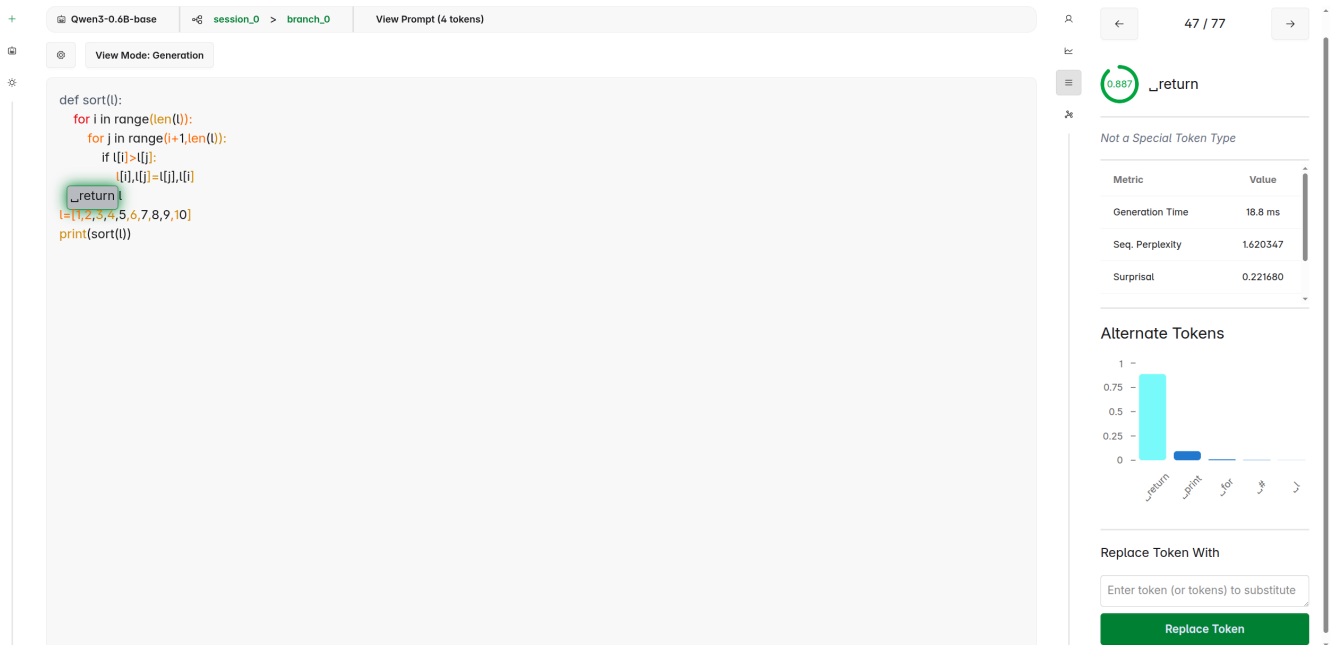


Figure 10: The Token Details sidebar presents special token types, token metrics and alternative tokens, and allows for creating new branches in the generation.

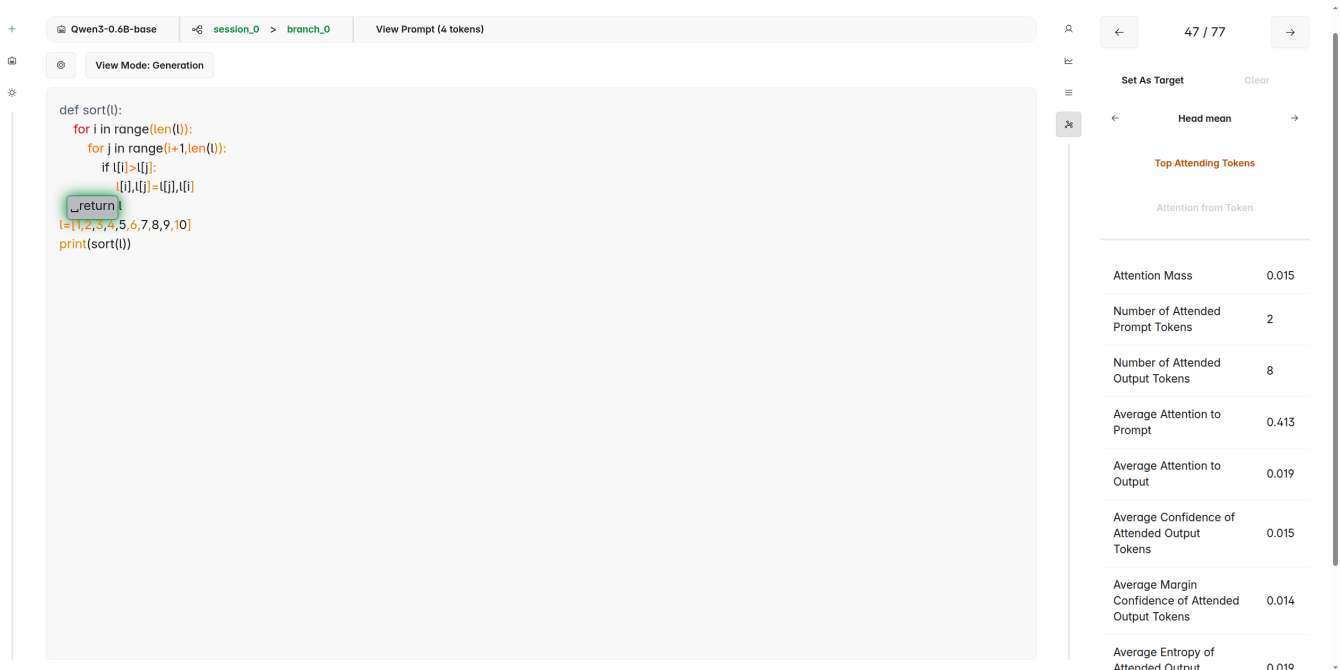
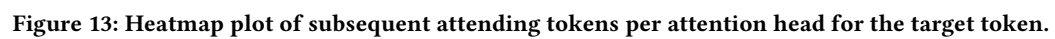
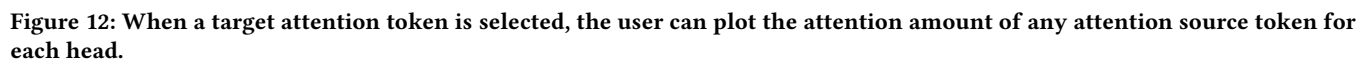


Figure 11: The Attention Details sidebar displays tokens' attention-relevant metrics, attention source tokens and subsequent attending tokens.



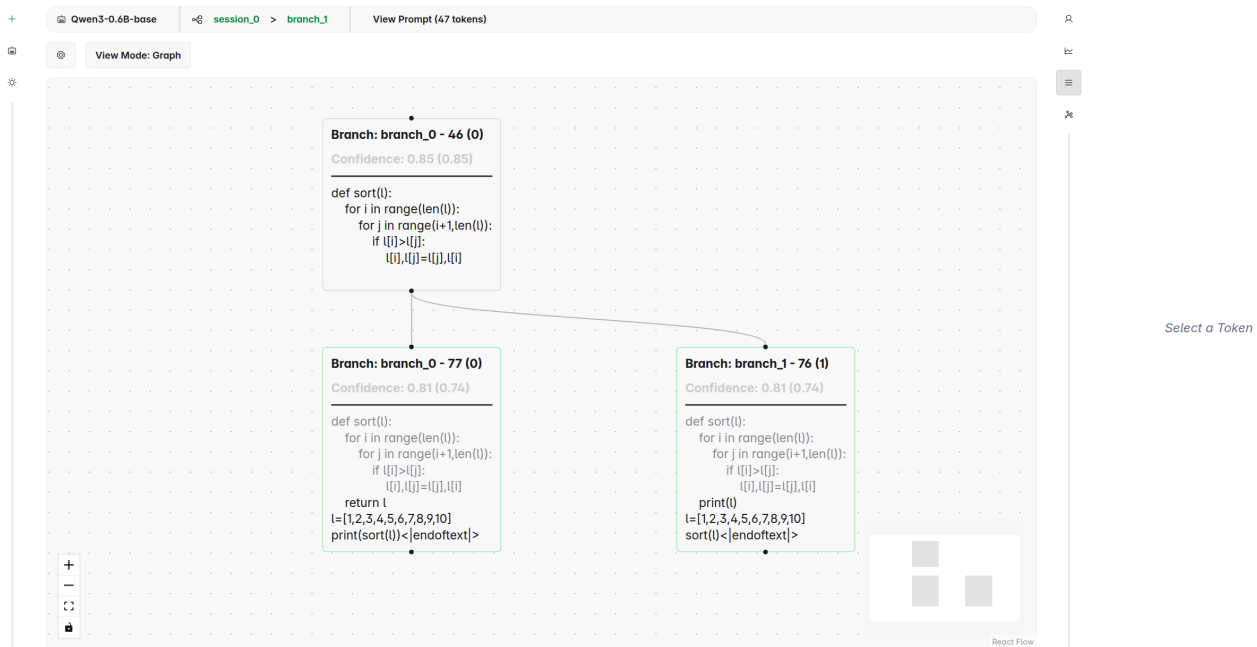


Figure 14: Graph view provides an overview of all branches of the generation and a summary of each branch.

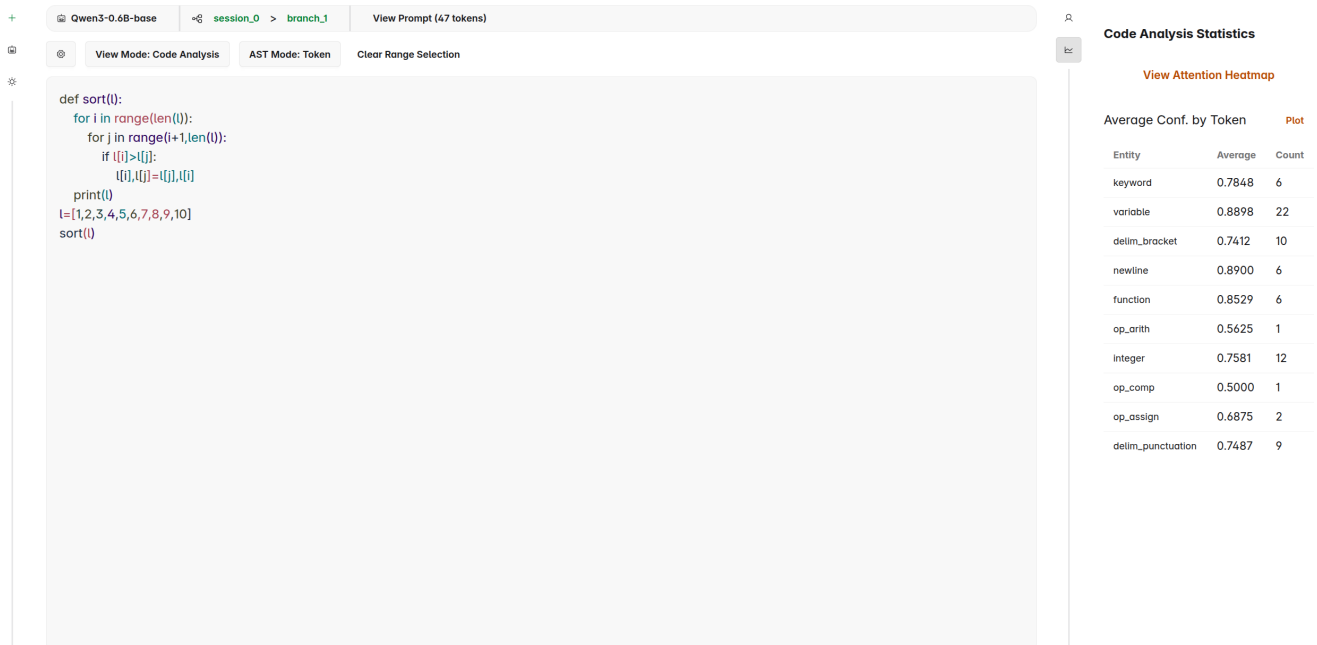


Figure 15: Code Analysis view displays extracted code entities and their average confidence scores.

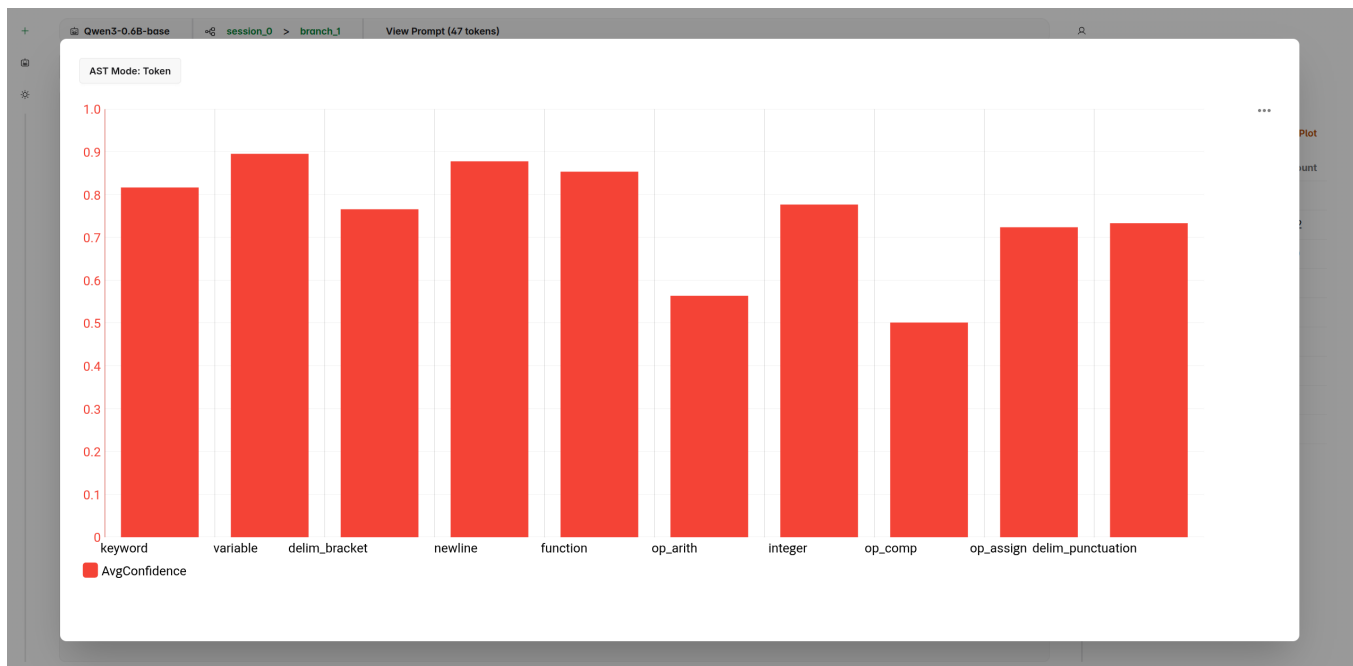


Figure 16: Plot view of the average confidence scores across the extracted code entities.

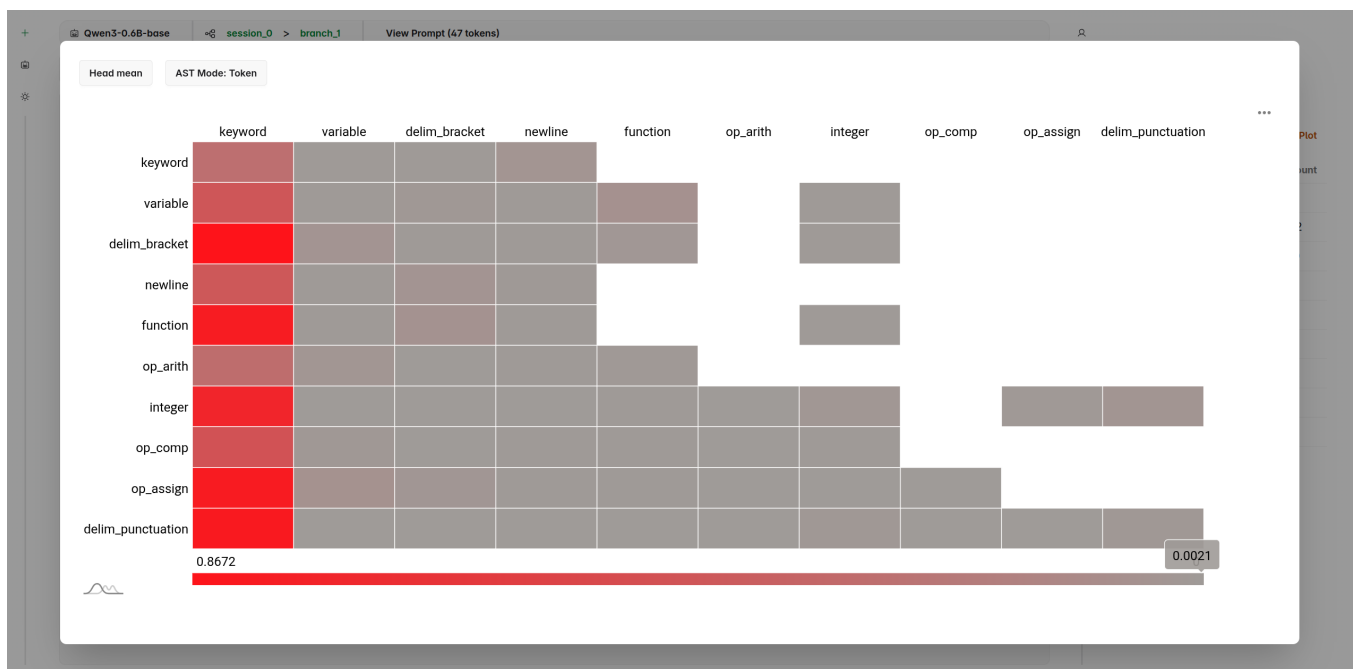


Figure 17: Heatmap of attention relationship between code entities for each attention head.